ESP32 UWB Pro with Display

Annotated Source Code · DW1000 Tag Firmware

This document presents the full source code for the ESP32 UWB Pro Tag sketch, with inline comments explaining every section. The board uses a DW1000 UWB radio for centimetre-accurate ranging against fixed anchors, and displays the nearest distance on a 128×64 SSD1306 OLED. Edit the `TAG_ADDR` constant and the `startAsTag` mode to customise behaviour.

■ File Header & Overview

```
/*
 * ESP32 UWB Pro with Display – TAG firmware
 *
 * This sketch runs on an ESP32 paired with a DW1000 UWB radio module and a
 * 128×64 SSD1306 OLED display.  The board acts as a UWB *tag*: it ranges
 * against one or more *anchors* and shows the distance (and RX power) of
 * the nearest anchor on the OLED.
 */
```

■ Includes & Libraries

```
#include <SPI.h>           // Hardware SPI bus used to talk to the DW1000
#include "DW1000Ranging.h" // Makerfabs/thotro DW1000 ranging library
#include <Wire.h>          // I²C bus used by the OLED display
#include <Adafruit_GFX.h>          // Core Adafruit graphics primitives
#include <Adafruit_SSD1306.h>      // Driver for the SSD1306 128×64 OLED
```

■ Configuration Constants

```
// Unique 8-byte EUI-64 address that identifies THIS tag on the UWB network.
// Every tag/anchor must have a different address.
#define TAG_ADDR "7D:00:22:EA:82:60:3B:9B"

// Uncomment to enable verbose serial debug prints inside the link-list helpers.
// #define DEBUG

// SPI bus pin assignments for the DW1000
#define SPI_SCK  18   // SPI clock
#define SPI_MISO 19   // Master-in / slave-out
#define SPI_MOSI 23   // Master-out / slave-in

// DW1000 control pins
#define UWB_RST 27    // Active-low hardware reset
#define UWB_IRQ 34    // Interrupt request line (signals new range data)
```

```
#define UWB_SS  21    // SPI chip-select (active low)
// I²C bus pin assignments for the OLED
#define I2C_SDA 4
#define I2C_SCL 5
```

■ Data Structures

```
// Singly-linked list node representing one discovered UWB anchor.
// A new node is appended whenever the DW1000 library calls newDevice().
struct Link {
    uint16_t anchor_addr; // 16-bit short address of the anchor
    float    range;       // Most recent range measurement (metres)
    float    dbm;         // Most recent RX power (dBm)
    struct Link *next;    // Pointer to next node (NULL = end of list)
};
// Head sentinel node – never holds real data; real anchors start at head->next.
struct Link *uwb_data;
// OLED display object: 128 px wide, 64 px tall, on the Wire I²C bus.
// The '-1' means no external RESET pin is wired.
Adafruit_SSD1306 display(128, 64, &Wire;, -1);
```

■ setup() – One-time Initialisation

```
void setup() {
    Serial.begin(115200);   // Open serial console for debug output
    // Initialise I²C with the custom SDA/SCL pins defined above
    Wire.begin(I2C_SDA, I2C_SCL);
    delay(1000);  // Give the OLED time to power up
    // Start the SSD1306 driver.  0x3C is the default I²C address.
    // Halts with an infinite loop if the display is not found.
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        Serial.println(F("SSD1306 allocation failed"));
        for (;;);
    }
    display.clearDisplay();
    logoshow();   // Show a splash screen for 2 seconds
    // Initialise the SPI bus, then hand the three DW1000 control pins
    // to the ranging library (Reset, ChipSelect, IRQ order).
    SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI);
    DW1000Ranging.initCommunication(UWB_RST, UWB_SS, UWB_IRQ);
    // Register callbacks – the library calls these automatically:
    DW1000Ranging.attachNewRange(newRange);         // New distance measurement ready
    DW1000Ranging.attachNewDevice(newDevice);       // A new anchor has been discovered
    DW1000Ranging.attachInactiveDevice(inactiveDevice); // An anchor has gone silent
    // Start ranging in TAG mode.  MODE_LONGDATA_RANGE_LOWPOWER trades
```

```
    // speed for maximum range and minimal power consumption.
    DW1000Ranging.startAsTag(TAG_ADDR, DW1000.MODE_LONGDATA_RANGE_LOWPOWER);
    uwb_data = init_link();  // Allocate the head sentinel for the anchor list
}
```

■ loop() – Main Event Loop

```
long int runtime = 0;  // Timestamp of the last display refresh (ms)
void loop() {
    // Must be called every iteration; drives the DW1000 state machine and
    // fires the newRange / newDevice / inactiveDevice callbacks when needed.
    DW1000Ranging.loop();
    // Refresh the OLED at most once per second to avoid flicker.
    if ((millis() - runtime) > 1000) {
        display_uwb(uwb_data);
        runtime = millis();
    }
}
```

■ DW1000 Callbacks

```
// Called by the library each time a fresh distance measurement arrives.
void newRange() {
    Serial.print("from: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getShortAddress(), HEX);
    Serial.print("\t Range: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getRange());
    Serial.print(" m");
    Serial.print("\t RX power: ");
    Serial.print(DW1000Ranging.getDistantDevice()->getRXPower());
    Serial.println(" dBm");
    // Update the matching node in the linked list with the latest values.
    fresh_link(uwb_data,
               DW1000Ranging.getDistantDevice()->getShortAddress(),
               DW1000Ranging.getDistantDevice()->getRange(),
               DW1000Ranging.getDistantDevice()->getRXPower());
}
// Called when the DW1000 library first detects a new anchor.
// Adds a new node to the linked list so we start tracking it.
void newDevice(DW1000Device *device) {
    Serial.print("ranging init; 1 device added ! -> ");
    Serial.print(" short:");
    Serial.println(device->getShortAddress(), HEX);
    add_link(uwb_data, device->getShortAddress());
}
```

```
// Called when an anchor has not responded for a timeout period.
// Removes its node from the list to keep the display clean.
void inactiveDevice(DW1000Device *device) {
    Serial.print("delete inactive device: ");
    Serial.println(device->getShortAddress(), HEX);
    delete_link(uwb_data, device->getShortAddress());
}
```

■ Linked-List Helpers

```
// Allocate and return an empty head sentinel node.
// The sentinel simplifies insertion/deletion (no special-case for empty list).
struct Link *init_link() {
    struct Link *p = (struct Link *)malloc(sizeof(struct Link));
    p->next = NULL;
    p->anchor_addr = 0;
    p->range = 0.0;
    return p;
}
// Append a new anchor node with the given short address to the end of the list.
void add_link(struct Link *p, uint16_t addr) {
    struct Link *temp = p;
    while (temp->next != NULL)      // Walk to the last node
        temp = temp->next;
    Serial.println("add_link: find struct Link end");
    struct Link *a = (struct Link *)malloc(sizeof(struct Link));
    a->anchor_addr = addr;
    a->range = 0.0;
    a->dbm   = 0.0;
    a->next  = NULL;
    temp->next = a;   // Attach new node at the end
}
// Search the list for a node matching 'addr'.
// Returns a pointer to the node, or NULL if not found.
struct Link *find_link(struct Link *p, uint16_t addr) {
    if (addr == 0) {
        Serial.println("find_link: Input addr is 0"); return NULL;
    }
    if (p->next == NULL) {
        Serial.println("find_link: Link is empty"); return NULL;
    }
    struct Link *temp = p;
    while (temp->next != NULL) {
        temp = temp->next;
        if (temp->anchor_addr == addr)
            return temp;
```

```
    }
    Serial.println("find_link: Can't find addr");
    return NULL;
}
// Update the range and RX-power fields of the node matching 'addr'.
void fresh_link(struct Link *p, uint16_t addr, float range, float dbm) {
    struct Link *temp = find_link(p, addr);
    if (temp != NULL) {
        temp->range = range;
        temp->dbm   = dbm;
    } else {
        Serial.println("fresh_link: Fresh fail");
    }
}
// Remove and free the node matching 'addr'.
// Uses the classic "track previous node" deletion pattern.
void delete_link(struct Link *p, uint16_t addr) {
    if (addr == 0) return;
    struct Link *temp = p;
    while (temp->next != NULL) {
        if (temp->next->anchor_addr == addr) {
            struct Link *del = temp->next;
            temp->next = del->next;   // Bypass the deleted node
            free(del);
            return;
        }
        temp = temp->next;
    }
}
// Debug helper – dumps the entire list to the serial console.
void print_link(struct Link *p) {
    struct Link *temp = p;
    while (temp->next != NULL) {
        Serial.println(temp->next->anchor_addr, HEX);
        Serial.println(temp->next->range);
        Serial.println(temp->next->dbm);
        temp = temp->next;
    }
}
```

■ OLED Display Functions

```
// Show a brief Makerfabs / DW1000 DEMO splash screen at startup.
void logoshow(void) {
    display.clearDisplay();
    display.setTextSize(2);
```

```
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0);
    display.println(F("Makerfabs"));
    display.setTextSize(1);
    display.setCursor(0, 20);
    display.println(F("DW1000 DEMO"));
    display.display();
    delay(2000);   // Hold the splash for 2 seconds
}
// Render the current ranging data from the linked list onto the OLED.
// Only the first anchor in the list is displayed (fits 128×64 at text size 2).
void display_uwb(struct Link *p) {
    struct Link *temp = p;
    int row = 0;
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    // If no anchors are tracked, show a "No Anchor" message.
    if (temp->next == NULL) {
        display.setTextSize(2);
        display.setCursor(0, 0);
        display.println("No Anchor");
        display.display();
        return;
    }
    while (temp->next != NULL) {
        temp = temp->next;
        char c[30];
        // Line 1 – distance in metres (1 decimal place)
        sprintf(c, "%.1f m", temp->range);
        display.setTextSize(2);
        display.setCursor(0, row++ * 32);  // Each row is 32 px tall at size 2
        display.println(c);
        display.println("");
        // Line 2 – received signal strength in dBm (2 decimal places)
        sprintf(c, "%.2f dbm", temp->dbm);
        display.setTextSize(2);
        display.println(c);
        // Stop after the first anchor – screen is too small for more at this size.
        if (row >= 1) break;
    }
    delay(100);        // Short pause before committing the frame buffer
    display.display(); // Push the buffer to the physical OLED
}
```

End of annotated source. Hardware: Makerfabs ESP32 UWB Pro · Library: DW1000Ranging / thotro